

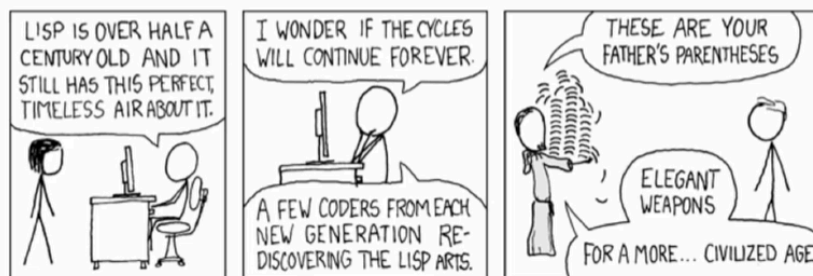
Lecture 27. Scheme

Scheme

Scheme is a Dialect of Lisp

What are people saying about Lisp?

- "The greatest single programming language ever designed."
– Alan Kay, co-inventor of Smalltalk and OOP
- "The only computer language that is beautiful."
– Neal Stephenson, DeNero's favorite sci-fi author
- "God's programming language."
– Brian Harvey, Berkeley CS instructor extraordinaire



Fundamentals

expressions:

- Primitive expressions: 2, 3.3, true, +, quotient,
- Combinations: (quotient 10 2), (not true),

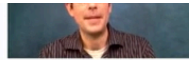
Numbers are self-evaluating; symbols are bound to values

Call expressions include an operator and 0 or more operands in parentheses

```
> (quotient 10 2)
5
> (quotient (+ 8 7) 5)
3
> (+ (* 3
      (+ (* 2 4)
          (+ 3 5)))
      (+ (- 10 7)
          6))
```

"quotient" names Scheme's built-in integer division procedure (i.e., function)

Combinations can span multiple lines (spacing doesn't matter)



```

scm> (*)
1
scm> (* 2 2 2 2 2 3 3 3)
Traceback (most recent call last):
Error: invalid numeral or symbol: 3}
scm> (* 2 2 2 2 2 3 3 3)
864
scm> (- (* 2 2 2 2 2 3 3 3) 1)
863
scm> (/ (+ (* (- (* 2 2 2 2 2 3 3 3) 1) 7) 1) 3)
2014
scm> +
#[+]
scm> (number? 3)
#t
scm> (number? +)
#f
scm> (zero? 2)
#f
scm> (zero? 0)
#t
scm> (zero? (- 2 2))
#t
scm> (integer? 2.2)
#f
scm> (integer? 2)
#t
scm>

```



Special Forms

Special Forms



A combination that is not a call expression is a *special form*:

- If expression: (if <predicate> <consequent> <alternative>)
- And and or: (and <e₁> ... <e_n>), (or <e₁> ... <e_n>)
- Binding symbols: (define <symbol> <expression>)
- New procedures: (define (<symbol> <formal parameters>) <body>)

Evaluation:

- (1) Evaluate the predicate expression.
- (2) Evaluate either the consequent or alternative.

```

> (define pi 3.14)
> (* pi 2)
6.28

```

The symbol "pi" is bound to 3.14 in the global frame

```

> (define (abs x)
  (if (< x 0)
      (- x)
      x))

```

A procedure is created and bound to the symbol "abs"

```

> (abs -3)
3

```

implement a square root function with scheme

```

(define (average x y)
  (/ (+ x y) 2))

```

```

(define (square x)
  (* x x))

```

```

(define (sqrt x)
  (

```

```

        (define (update guess)
          (if (= (square guess) x)
              guess
              (average guess (/ x guess)))
        )
      )
    )
  (update 1)
)

```

Scheme Interpreters

in project 4 you implements a scheme interpreter

you will do that by actually writing the code that executes the different procedues in scheme.

Lambda Expressions

```
(lambda (<formal-parameters>) <body>)
```

```
# case
```

```
(define (plus4 x) (+ x 4))
```

```
(define plus4 (lambda (x) (+ x 4)))
```

Example: Sierpinski's Triangle

there are built-in drawing functions in scheme

```

(fd 100) # forward
(rt 90) # right-turn 90
(fd 40)
(bk 100)
(lt 90)
(define (line) (fd 50))
(line)
(line)
(rt 90)
(define (twice fn) (fn) (fn))
(twice line)

```

```
(load 'ex.scm)
```

```
# ex.scm
```

```
(define (line) (fd 50))
```

```

(define (twice fn) (fn) (fn))
(define (repeat k fn)
  (fn)
  (if (> k 1) (repeat (- k 1) fn))
)
(define (tri fn)
  (repeat 3 (lambda () (fn) (lt 120))))

(define (sier d k)
  (tri (lambda () (if (= d 1) (fd k) (leg d k)))))

(define (leg d k)
  (sier (- d 1) (/ k 2))
  (penup) (fd k) (pendown)
)

```

```

Welcome to the CS 61A Scheme Interpreter (version 1.2.
4)
scm> (rt 90)
scm> (speed 0)
scm> (sier 5 200)
Traceback (most recent call last):
  0 (sier 5 200)
  1 sier
Error: unknown identifier: sier
scm> (load 'ex.scm)

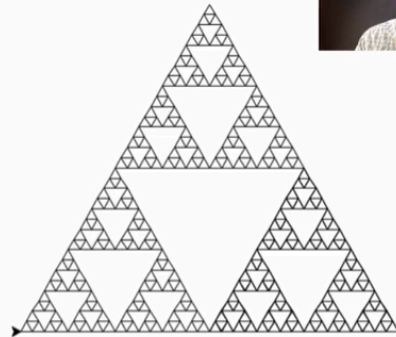
scm> (sier 5 200)
scm> (bk 200)
scm> (sier 6 400)
scm>

```

```

ex.scm ~/Documents/works.../61a_lectures/fa19 - VIM
(define (line) (fd 50))
(define (twice fn) (fn) (fn))
(define (repeat k fn)
  (fn)
  (if (> k 1) (repeat (- k 1) fn)))
(define (tri fn)
  (repeat 3 (lambda () (fn) (lt 120))))
(define (sier d k)
  (tri (lambda () (if (= d 1) (fd k) (leg d k)))))
(define (leg d k)
  (sier (- d 1) (/ k 2))
  (penup) (fd k) (pendown))
~
~
~
~

```



Cond

The cond special form that behaves like if-elif-else statements in Python

```

(cond ((x > 10) (print 'big))
      ((x > 5) (print 'medium))
      (else (print 'small'))
)

```

Begin

normally, only one expression will be allowed and evaluated, however, the begin special form combines multiple expressions into one expression

```
(begin (print 'big) (print 'guy))
```

Let

The let special form binds symbols to values temporarily; just for one expression

case

in python

```
a = 3
b = 2 + 2
c = math.sqrt(a*a + b*b)
```

the values are bind to the variable for more the one sentence

in scheme

```
(define c (let ((a 3) (b (+ 2 2)))
            (sqrt (+ (* a a) (* b b)))))
))
```

The difference is that after this sentence, values will no longer be binded to a and b

Lists

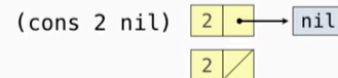
that's why we learn the linked list in that form, because it's similar to the ones in scheme

Scheme Lists



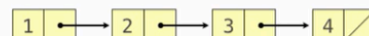
In the late 1950s, computer scientists used confusing names

- **cons**: Two-argument procedure that creates a linked list
- **car**: Procedure that returns the first element of a list
- **cdr**: Procedure that returns the rest of a list
- **nil**: The empty list



Important! Scheme lists are written in parentheses with elements separated by spaces

```
> (cons 1 (cons 2 nil))
(1 2)
> (define x (cons 1 (cons 2 nil)))
> x
(1 2)
> (car x)
1
> (cdr x)
(2)
> (cons 1 (cons 2 (cons 3 (cons 4 nil))))
(1 2 3 4)
```



Symbolic Programming

important feature: QUOTATION

```
scm> (car (cdr (car (cdr '(1 (2 3) 4)))))  
3  
scm> (car (cdr (car (cdr '(a (b c) d)))))  
c
```

Symbolic Programming

Symbols normally refer to values; how do we refer to symbols?

```
> (define a 1)  
> (define b 2)  
> (list a b)  
(1 2)
```

No sign of "a" and "b" in the resulting value

Quotation is used to refer to symbols directly in Lisp.

```
> (list 'a 'b)  
(a b)  
> (list 'a b)  
(a 2)
```

Short for (quote a), (quote b):
Special form to indicate that the expression itself is the value.

Quotation can also be applied to combinations to form lists.

```
> '(a b c)  
(a b c)  
> (car '(a b c))  
a  
> (cdr '(a b c))  
(b c)
```

Program as Data

A Scheme Expression is a Scheme List

Scheme programs consist of expressions, which can be:

- Primitive expressions: 2 3.3 true + quotient
- Combinations: (quotient 10 2) (not true)

The built-in Scheme list data structure (which is a linked list) can represent combinations

```
scm> (list 'quotient 10 2)  
(quotient 10 2)  
  
scm> (eval (list 'quotient 10 2))  
5
```

In such a language, it is straightforward to write a program that writes a program

case

```
(define (fact n)  
  if (= n 0) 1 (* n fact(- n 1)))
```

```
(define (exp-fact n)
  if (= n 0) 1 (list '* n fact(- n 1)))
```

Generating Code

people like this kind of programming languages because it's easy for us to write code to generate code with this language!

Quasiquotation

- **quotation:** `'(...)` (也就是 `(quote ...)`) 把后面的表达式当作**数据**返回，不会求值。
- **quasiquotation:** ``(...)` (也就是 `(quasiquote ...)`) 也基本把东西当作数据，但**允许**你在里面“开洞”把某些部分求值并插进去，用：
 - **unquote:** `,` (即 `(unquote ...)`) 插入一个值
 - **unquote-splicing:** `,@` (即 `(unquote-splicing ...)`) 把一个列表的元素“摊开”并入外层列表

Example: Implementation of While Statements

Example: While Statements

What's the sum of the squares of even numbers less than 10, starting with 2?

```
x = 2
total = 0
while x < 10:
    total = total + x * x
    x = x + 2
```

```
(begin
  (define (f x total)
    (if (< x 10)
        (f (+ x 2) (+ total (* x x)))
        total))
  (f 2 0)))
```

What's the sum of the numbers whose squares are less than 50, starting with 1?

```
x = 1
total = 0
while x * x < 50:
    total = total + x
    x = x + 1
```

```
(begin
  (define (f x total)
    (if (< (* x x) 50)
        (f (+ x 1) (+ total x))
        total))
  (f 1 0)))
```

```
(define (sum-while initial-x condition add-to-total update-x)
  (begin
    (define (f x total)
      (if ,condition
          (f ,update-x (+ total ,add-to-total))
          total))
    (f ,initial-x 0)))
```

